

Katabasis

Katabasis (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): *Katabasis* is a level-based macabre-themed RPG where players control three characters in **turn-based battles**. At the start of each battle, players assign a **card** to each character granting offensive/defensive abilities; then, based on each character's individual **action value**, players **tap and drag from each character across enemies to chain attacks**. Named technical features: a **decision-tracking enemy AI** that attacks automatically, evaluates the player's likely next action and counters it to force optimal play; a **level editor** for asset placement and building scenes; a **custom particle-effect (VFX) inspector** (spawn rate, color-over-time, scale); and a **gesture input system** (swipes, multi-finger triggers). Load-bearing C++: a **real-time combat simulation loop** holding a stable update rate across multiple enemies, AI behaviours, environment, repeated attacks and effects on target Android devices; and **static memory allocation & pooling** (a fixed number of enemies/particles/assets allocated at level load and recycled). Tools: a **level editor** (scenes, parallax, show/hide images), a **particle-effect inspector** (sliders for particle patterns), and an **asset importer & optimization pipeline** (import .mp3/.png etc., convert to engine format, compress). Built for Android — C++ rendering via OpenGL ES, connected to a Kotlin app through JNI. Design features: ≥ 5 turn-based combat encounters with designed enemy layouts; ≥ 5 ability cards; ≥ 3 deployable units with individual action values; and the tap-and-drag chain-attack system.

Note (module completeness): the proposal doesn't mention an **External Database** or accounts, but the module mandates Firebase as the external DB — plan to persist progress / unlocked cards / level records there (a standard, small addition; see the External Database row + M1G12-style integration). Not a category concern; just build it in.

References:

- Slay the Spire on Steam (the benchmark card-driven turn-based combat RPG — assign cards that grant abilities, then resolve attacks against a lineup of enemies — the card-ability-per-turn precedent)
- Darkest Dungeon on Steam (a macabre/gothic turn-based party RPG — lead a small party through level-based encounters with an innovative strategic turn system — the macabre-theme + party-turn-based precedent)
- Honkai: Star Rail — "Follow-Up Attack" (a turn-based combat system where each character's **action value** governs turn order and abilities chain **follow-up attacks** across enemies — the action-value + chain-attack precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Katabasis
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — C++ via JNI rendering through OpenGL ES, Kotlin app shell; device: Pixel 9a) — the tap-and-drag chain and gesture input are touch-first; the Native PC build is the development/tools vehicle (the level editor, particle inspector and asset importer). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for accounts and progress/card-unlock persistence.

Front end: A 2D macabre renderer — the three characters, the enemy layouts, parallax scenes, the card-assignment UI, the action-value/turn HUD, the tap-and-drag chain-target overlay, and the particle VFX. **Languages:**

Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the realtime 2D in a GLSurfaceView; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: The real-time C++ combat simulation loop (turn/action-value resolution, card abilities, chain-attack targeting, the decision-tracking enemy AI, particle updates), with static allocation + object pooling for enemies/particles/assets; plus Firebase for accounts + progress persistence.

Languages: C++ (engine); Firebase-backed services on the Extended tier.

Backend scope: Local now, server-ready (T2) — the accounts / progress-persistence seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — it resolves the action-value turn order, applies card abilities, executes the tap-and-drag chain attacks, runs the decision-tracking enemy AI and updates particles, holding a stable rate on Android with static allocation + pooling.
- *Data-driven:* cards (≥ 5), characters/units (≥ 3 , with action values), enemies and their layouts, encounters (≥ 5) and particle-effect definitions are all data — a new card, enemy or encounter is a data change (authored via the editor tools).
- *2D:* the macabre renderer, parallax scenes, the chain-target overlay, the card/turn UI and the particle VFX (custom-rendered). *3D:* not needed.
Networking: single-player; accounts/progress persistence is async client↔cloud (Application Communication ✓), so a Networking System is optional (●).
- *Custom tool:* three — the **level editor** (scenes/parallax/asset placement), the **particle-effect inspector** (spawn rate / color-over-time / scale sliders), and the **asset importer & optimization pipeline** (import + convert + compress).
- *External DB:* progress, unlocked cards and level records (Firebase — add this per the module). *Auth* »: optional accounts. *Analytics* »: optional — encounter win/loss + card usage for balancing.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the combat loop (action-value turns) + the gesture input system.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a real-time C++ loop resolving action-value turn order, card abilities and the tap-and-drag chain, with a gesture input system (swipes, multi-finger triggers); C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the enemy AI (M1G8) and particles (M1G13) run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Entity/component model with cards, characters/units (+ action values), enemies + layouts, encounters and particle-effect definitions in hot-reloadable JSON — a new card/enemy/encounter is a data change.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (the level editor), M2G13 (card/chain systems), M1G8 (enemy/AI data).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays (turn/action-value state, AI decisions, pool usage), Tracy profiling of a full encounter with many enemies + particle bursts to hold a stable rate on Android, Catch2 tests for the turn/chain resolution and data loading, on a GitHub Actions Windows+Android matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the static-pooling budget M2G1 validates.
- **G4 2D Graphics** — the macabre renderer + parallax scenes + the chain-target overlay.
Tech & dependencies
 - **Tech: Web build:** Sprite/tile renderer (characters, enemies, parallax backgrounds), a camera, batching, the tap-and-drag chain overlay and the card/turn HUD; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType.
 - **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** G1 (loop & window).
 - **Feeds into:** M2G4 (advanced VFX/lighting); every visible frame; the level editor shares this renderer.
- **G8 Enemy AI & Pathfinding** — the **decision-tracking enemy AI** (evaluate the player's likely next action and counter it).
Tech & dependencies
 - **Tech:** A decision model that scores the player's available actions, predicts the highest-value one and picks a counter (minimax/heuristic search over the turn state), auto-attacking and adapting per turn; tuned from data (custom C++).
 - **Prerequisites:** G1 (tick, turn state), G2 (enemy/ability data).
 - **Feeds into:** M2G10 (advanced AI + boss behaviour); the "forces optimal play" design pillar.
- **G13 Particle & VFX System** — the custom particle system behind the VFX inspector (spawn rate, color-over-time, scale), object-pooled.
Tech & dependencies
 - **Tech:** A pooled particle system (emitters, forces, color/scale-over-lifetime) driven by data so the inspector's sliders map to particle parameters, sized to hold a stable rate under combat bursts.
 - **Prerequisites:** G1 (loop), G4 (rendering).
 - **Feeds into:** M2G6 (the particle inspector tool); the attack/ability VFX.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — static allocation + pooling and on-device validation.

Tech & dependencies

- **Tech: Web build:** Static allocation + object pools for enemies/particles/assets (allocated at level load, recycled in play), Tracy profiling of a heavy encounter, ASan on MSVC, and on-device validation (Pixel 9a); the Firebase/Linux-server seam (accounts + progress) set up (account + software + basic connectivity test) by M2.
- Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable + a stable rate under particle-heavy combat.

- **G2 ★ Content Editor — Core (Level Editor)** — build scenes: asset placement, parallax layers, show/hide images.

Tech & dependencies

- **Tech:** Dear ImGui + the shared renderer: place assets, arrange parallax layers, toggle image visibility and lay out enemy encounters, with round-trip JSON serialization, live preview and undo/redo.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (preview).
- **Prerequisites:** G2 (scene/encounter data), M1G4 (renderer).
- **Feeds into:** All level content; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android Debug builds, tests (turn/chain resolution, AI, data loading) and packaging via GitHub Actions; the asset importer/optimization pipeline can run as a CI step. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI & test foundation).

- **Feeds into:** Release/showcase builds of every goal.

- **G6 Content Editor — Advanced Tool Features (Particle VFX Inspector)**
— sliders for spawn rate, color-over-time and scale, with live preview.

Tech & dependencies

- **Tech:** Dear ImGui inspector exposing the M1G13 particle parameters as sliders (spawn rate, color-over-time gradient, scale curve), with live preview and JSON save/load of effect presets.
 - **Prerequisites:** M1G13 (the particle system), M2G2 (editor patterns).
 - **Feeds into:** Fast VFX authoring without code; the content showcase.
- **G10 AI — Advanced** — the decision-tracking counter-AI matured (deeper look-ahead, boss/enemy tiers).

Tech & dependencies

- **Tech:** Behaviour trees + deeper heuristic/minimax look-ahead over the turn state for the counter-AI, difficulty tiers and distinct enemy roles, tuned from data.
 - **Prerequisites:** M1G8 (the decision-tracking AI foundation).
 - **Feeds into:** The ≥ 5 encounters + any boss; the "optimal play" pressure at showcase.
- **G13 Gameplay Systems II** — the card / action-value / **tap-and-drag chain-attack** system matured.

Tech & dependencies

- **Tech:** C++ systems over the ECS for the ≥ 5 card abilities (offensive/defensive), the per-unit action-value economy governing chain frequency, and the tap-and-drag chain resolution across enemies; balancing-data-driven.
- **Prerequisites:** M1G2 (card/unit data), M1G1 (chain input/loop).
- **Feeds into:** The core combat depth at showcase; T2 card/encounter expansion.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 2 Art (9) — a macabre RPG reads by its **characters, enemies, cards and scenes**: three characters, ≥ 5 enemies with designed layouts, ≥ 5 card illustrations, macabre parallax environments and the attack VFX are a large art surface, so two artists carry it; a single designer owns the card-ability design, the action-value/chain-attack systems, the ≥ 5 encounters and the counter-AI difficulty design (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the card/encounter/balance systems outgrow one designer, trade the 2nd artist for a 2nd designer. *Scope note (advisory):* a real-time combat loop + decision-tracking AI + card/chain systems + three editor tools + pooling is a full but well-structured scope — the action-value turn combat with the chain attack, ~ 3 cards, the counter-AI and the level editor is the M1/M2 floor; the full card set, all encounters, the particle inspector and the asset pipeline are the stretch.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the card + action-value + chain-attack combat, the macabre theme and the level-based encounter structure; primary benchmark e.g. Slay the Spire / Honkai: Star Rail, with a Singapore benchmark for the local-awareness criterion)
- M1D2 Core Gameplay Loop Prototype (a greybox proving assign-card → action-value turn → tap-and-drag chain is satisfying) · prototype in Unity
- M1D6 Systems & Economy Design (the ≥ 5 card abilities, the per-unit action-value economy governing chain frequency, and the enemy-layout/encounter design — every system with a purpose)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (card/action-value/enemy balance and the counter-AI difficulty via the team's JSON, no code changes)

- **UI/UX M1** (graded by the Design instructor for the Games category):
 - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the card-assignment UI, the action-value/turn order and the tap-and-drag chain-target affordance.
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — battle, card-assignment, chain-target and results screens navigable unaided on the Native-Android target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished combat + chain UX tested with players on the Pixel 9a.
- **Art M1:** 2 artists justified (3 characters + enemy roster + cards + macabre scenes + VFX)
 - M1A1 ★ Art Bible / Proof of Concept (the macabre visual direction — palette, tone, card frame language)
 - M1A2 Character Concepts & Model Sheets (the 3 playable characters + the ≥ 5 enemy types)
 - M1A3 Environment, Props & VFX Concepts (the macabre parallax scenes + the attack/ability VFX concepts)
 - M1A4 UI Art Concepts & Typography (card art/frames, action-value/turn HUD, battle UI)
 - M1A5 Animation Foundations (character attack/chain, enemy idle/attack, hit reactions)
 - M1A6 Early Asset Production — Prototype Support
- **Art M2:**
 - M2A1 ★ Critical Asset Production & In-Engine Visual Quality
 - M2A2 Character Animation — Extensive (character/enemy attack, chain, counter and death animations)
 - M2A3 Environment Alive — Ambient Animation & Layering (living macabre parallax scenes)
 - M2A4 UI Art — States & Feedback (card-assigned / chain-target / counter states)
 - M2A5 VFX — Critical Effects & Transitions (the card/ability effects, chain-attack impacts — authored in the particle inspector)
 - M2A6 Cutscenes & Storyboards (the macabre level intros / story beats)

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (macabre BGM, card-assign, attack, chain-hit, counter and defeat cues), suitable and correctly triggered.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for overlapping combat SFX.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (macabre BGM + card / attack / chain / counter / enemy / defeat SFX, ≥6 types), scaling with combat intensity.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; a distinct chain/counter cue).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the level editor + particle inspector), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*